

Backtracking Fundamentals

1 Introduction

Definition

Backtracking is a systematic search technique that builds a solution one decision at a time, checks whether the partial solution can still become valid, and abandons a branch as soon as it becomes impossible.

Backtracking is refined brute force: it checks partial candidates while they are being built instead of waiting until a full candidate is complete.

Core Pattern

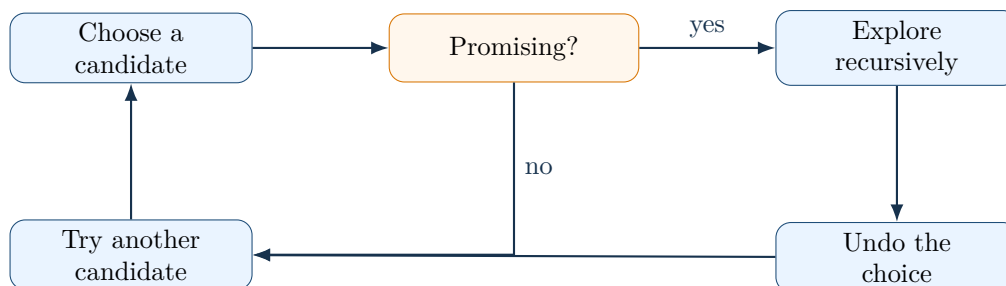
The essential cycle is:

Choose → Explore → Undo → Try another choice

Here the focus is on feasible-solution search, not optimisation.

2 Backtracking Control Flow

At each recursive level, the algorithm chooses a candidate, tests it, explores it if valid, and then undoes the choice.



Key Point

The undo step restores the previous partial solution before the next candidate is tried.

3 Brute Force vs Backtracking

Aspect	Brute Force	Backtracking
Checking	After complete candidates.	During construction.
Search space	Often explores everything.	Prunes bad branches early.
Traversal	Exhaustive generation.	Usually depth-first search.

Main Contrast

Brute force checks full candidates. Backtracking rejects impossible partial candidates.

4 State-Space Tree

State-Space Tree

A **state-space tree** represents all possible decisions of a search problem.

- The root represents the empty solution.
- Each internal node represents a partial solution.
- Each edge represents a choice.
- Each leaf is a solution or a dead end.

Backtracking walks this tree depth first and cuts off nonpromising subtrees.

Promising Test

Let S_k be the partial solution after k decisions.

$$P(S_k) = \begin{cases} \text{true}, & \text{if } S_k \text{ may still become valid,} \\ \text{false}, & \text{if } S_k \text{ cannot become valid.} \end{cases}$$

Backtracking explores S_k only when

$$P(S_k) = \text{true.}$$

5 Worked Arrangement Example

Problem

Arrange three people

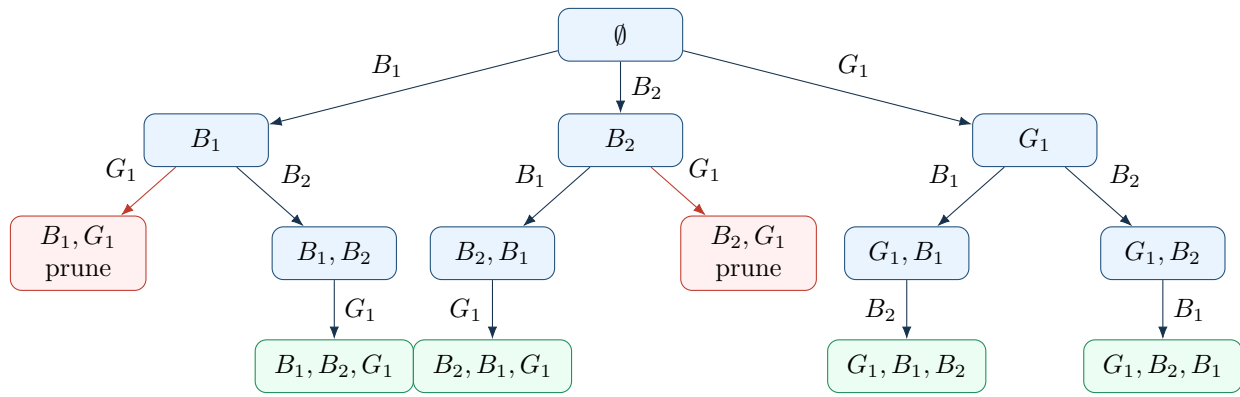
$$B_1, \quad B_2, \quad G_1$$

in three positions, subject to the condition that G_1 must not sit in the middle position.

Total possible arrangements:

$$3! = 3 \cdot 2 \cdot 1 = 6$$

Backtracking prunes a branch as soon as the second position becomes G_1 .



Solution

Forbidden partial state:

second position = G_1 .

Pruned branches:

$(B_1, G_1), \quad (B_2, G_1).$

Valid arrangements:

$G_1, B_1, B_2, \quad G_1, B_2, B_1, \quad B_1, B_2, G_1, \quad B_2, B_1, G_1.$

Answer

Backtracking avoids completing the branches where G_1 is already in the middle.

6 Generic Backtracking Template

Backtracking Template

```

Backtrack(level):
    if a complete solution has been formed:
        process or print the solution
        return

    for each candidate choice:
        make the choice

        if the current partial solution is promising:
            Backtrack(level + 1)

        undo the choice
  
```

7 Correctness Intuition

1. Every possible complete candidate can be reached by some sequence of choices in the state-space tree.

2. The promising function never rejects a partial solution that could still be extended into a valid complete solution.

So every valid solution remains reachable, while impossible branches are skipped.

8 Running Time

If there are at most b choices per level and the solution length is n , the full search tree may contain

$$1 + b + b^2 + \cdots + b^n = \frac{b^{n+1} - 1}{b - 1}$$

nodes.

Worst-case time:

$$\boxed{\mathcal{O}(b^n)}$$

Why Backtracking Still Helps

Pruning can save large subtrees, but many backtracking problems are still exponential in the worst case.